

CS355 Lab 2 — Batch B (15:30–16:55)

Language: Scheme (DrRacket, `#lang sicp`)

Q1 (1 mark) — `curried-in-range` (file name: q1.scm)

Write a **curried** procedure `curried-in-range` that takes a lower bound `lo`, an upper bound `hi`, and a value `x` (one at a time, via nested `lambdas`, not Racket's multi-argument `define` shorthand), and returns whether `x` lies between `lo` and `hi` (inclusive).

```
((curried-in-range 1) 10) 5) => #t
((curried-in-range 1) 10) 15) => #f
((curried-in-range 5) 5) 5) => #t
```

Q2 (2 marks) — Church encodings, extended (file name: q2.scm)

You've seen (or will see) how booleans, numbers, and pairs can be built purely out of functions in lambda calculus. Assume the following definitions are already available to you — copy this block into your file as-is, don't redefine them:

```
(define ctrue (lambda (t) (lambda (f) t)))
(define cfalse (lambda (t) (lambda (f) f)))
(define cif (lambda (cb) (lambda (t) (lambda (f) ((cb t) f))))))
(define czero (lambda (f) (lambda (x) x)))
(define csucc (lambda (n) (lambda (f) (lambda (x) (f ((n f) x))))))
(define church->int (lambda (n) ((n (lambda (k) (+ k 1))) 0)))
(define ccons (lambda (a) (lambda (b) (lambda (sel) ((sel a) b)))))
(define ccar (lambda (p) (p (lambda (a) (lambda (b) a)))))
(define ccdr (lambda (p) (p (lambda (a) (lambda (b) b)))))
```

Firstly, make sure you understand how these procedures work. Next, your job is to build **new** things on top of these — using only `lambda`, function application, and the definitions above (`church->int` is for testing purposes only; don't use it, or Scheme's real `if`, inside the two answers below — they should stay purely in "Church form").

(a) `church-swap` (1 mark). Write `church-swap`, which takes a Church pair and returns a new Church pair with the first and second elements swapped.

```
(church->int (ccar (church-swap ((ccons c2) c3)))) => 3 ; where c2, c3
are Church numerals for 2, 3
(church->int (ccdr (church-swap ((ccons c2) c3)))) => 2
```

(b) church-is-even? (1 mark). Write `church-is-even?`, which takes a Church numeral and returns a Church boolean: "true" if the numeral is even, "false" otherwise. Hint: think about what happens if you flip a Church boolean's truth value once for every time the numeral's function gets applied — a "church-not" that flips a Church boolean might come in handy (you're free to define your own helpers).

```
(church-is-even? czero)           ; behaves like ctrue  
(church-is-even? (csucc czero)) ; behaves like cfalse
```

Once you're done, feel free to also try Batch A's Q1–Q2 for extra practice — same topics, different problems.